

A PROJECT REPORT

AttendEase: Classroom Attendance and Analytics Dashboard

System to mark and analyze attendance

Submitted by

Anshika Chauhan	(DL.AI.U4AID24106)
Kalyani Agarawal	(DL.AI.U4AID24114)
Madesh KK	(DL.AI.U4AID24120)
Parima Verma	(DL.AI.U4AID24125)

in partial fulfilment for the award of the degree of

**BACHELOR OF TECHNOLOGY (B. Tech) IN ARTIFICIAL
INTELLIGENCE AND DATA SCIENCE (AIDS)**

Under the Guidance of

Dr. Vijay Kumar Soni Dr. Divyanshu Sinha
(UID) (DBMS)

Submitted to



AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE
FARIDABAD – 121002

5th May 2026



BONAFIDE CERTIFICATE

This is to certify that this project report entitled “**AttendEase: Classroom Attendance and Analytics Dashboard System to mark and analyze attendance**” is the **bonafide work** of the following students, who carried out the project work under my supervision:

- **Anshika Chauhan** (DL.AI.U4AID24106)
- **Kalyani Agarawal** (DL.AI.U4AID24114)
- **Madesh KK** (DL.AI.U4AID24120)
- **Parima Verma** (DL.AI.U4AID24125)

SIGNATURE

Dr. Vijay Kumar Soni Dr. Divyanshu Sinha

Assistant Professor

Assistant Professor

School of AI

School of AI

Amrita Vishwa Vidyapeetham, Faridabad

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled “**AttendEase: Classroom Attendance and Analytics Dashboard System to mark and analyze attendance**” submitted by us in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** is the record of the project work carried out by us under the guidance of **Dr. Vijay Kumar Soni (UID)** and **Dr. Divyanshu Sinha (DBMS)**. This work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or title in this or any other University or institution of higher learning.

SIGNATURES

Anshika Chauhan
DL.AI.U4AID24106

Kalyani Agarawal
DL.AI.U4AID24114

Madesh KK
DL.AI.U4AID24120

Parima Verma
DL.AI.U4AID24125

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution and support of many people. We take immense pleasure in expressing our profound gratitude to our honorable Chancellor, **Sri Mata Amritanandamayi Devi**, for her blessings and for being a constant source of inspiration.

We express our sincere gratitude to our Principal, **Dr. Lakshmi Mohandas**, Amrita School of Computing and Engineering, for providing the necessary facilities and continuous support to gain valuable education and learning experience.

We would like to express our heartfelt gratitude to our supervisors, **Dr. Vijay Kumar Soni (UID)** and **Dr. Divyanshu Sinha (DBMS)**, for their personal involvement, invaluable guidance, and constant encouragement throughout the course of this project work.

We are thankful to the Project Supervisor, Review Panel Members, and the entire faculty of the Department of Computer Science & Engineering for their constructive criticism and valuable suggestions, which have been a rich source for improving the quality of this work.

Anshika Chauhan
DL.AI.U4AID24106

Kalyani Agarawal
DL.AI.U4AID24114

Madesh KK
DL.AI.U4AID24120

Parima Verma
DL.AI.U4AID24125

ABSTRACT

AttendEase is a web-based Classroom Attendance and Analytics Dashboard designed to digitize and streamline the process of recording, tracking, and analyzing student attendance in academic institutions. The system replaces traditional paper-based or manual attendance methods with an intelligent, role-based digital platform that provides real-time insights to faculty, students, and administrators.

The system is built on a three-tier architecture: a MySQL relational database as the data layer, a Python Flask RESTful API as the backend logic layer, and a vanilla HTML/CSS/-JavaScript single-page application as the frontend presentation layer. Three distinct user roles — Faculty, Student, and Admin — each have a dedicated dashboard tailored to their specific responsibilities and access privileges.

Faculty members can define courses, assign sections, configure weekly schedules, enroll students, and mark attendance through an interactive roster interface. Students can monitor their own attendance percentage per course, view today's class schedule, receive low-attendance alerts, and interact with an AI-powered chatbot assistant. Administrators have system-wide visibility over all users, courses, sections, and settings.

The database schema consists of seven normalized relational tables — **faculty**, **student**, **section**, **course**, **enrollment**, **attendance**, and **schedule** — with enforced foreign key constraints ensuring referential integrity. Key SQL operations include attendance percentage computation using conditional aggregation, identification of students below the 75% threshold using **HAVING** clauses, and atomic batch attendance marking using **INSERT ... ON DUPLICATE KEY UPDATE** transactions.

Additional features include a live clock displaying real-time date, day, and time across all pages, a role-aware timetable showing today's scheduled lectures, an analytics dashboard with bar charts and donut visualizations, and a natural language chatbot capable of answering attendance queries using keyword-based intent detection backed by live database queries.

The system demonstrates practical application of database management concepts including normalization, multi-table joins, aggregation functions, transactions, and role-based data access — making it a comprehensive solution for academic attendance management.

Keywords: Attendance Management System, Web-based Application, Role-based Access Control, MySQL, Python Flask, RESTful API, Database Normalization, Conditional Aggregation, Analytics Dashboard, Single-Page Application, Academic Information System, Chatbot, Real-time Tracking, Multi-table Joins, Student Monitoring

Contents

List of Figures	8
List of Tables	8
1 Introduction	10
1.1 Background and Motivation	10
1.2 Problem Statement	11
1.3 Objectives	12
1.4 Scope of the System	13
2 Methodology	16
2.1 Development Approach	16
2.1.1 Phase 1 — Database Design and Schema Definition	16
2.2 Phase 2 — Backend API Development	20
2.2.1 Application Structure	21
2.2.2 Database Connection Design	21
2.2.3 Complete REST API Specification	21
2.2.4 Authentication Flow	22
2.2.5 Chatbot Intent Detection Algorithm	23
2.3 Phase 3 — Frontend UI Development	23
2.3.1 Design System	24
2.3.2 Page Architecture	24
2.3.3 Role-Based Navigation	24
2.3.4 Live Clock Implementation	24
2.3.5 Attendance Percentage Visualization	24
2.4 Phase 4 — Integration and Feature Completion	25
2.4.1 Data Loading Strategy	25
2.4.2 Timetable Integration	25
2.4.3 Threshold Alert System	25
2.5 Phase 5 — Testing and Validation	26
2.5.1 Database Testing	26
2.5.2 API Testing	26
3 Database Design	27
3.1 Entity-Relationship Diagram Description	27
3.1.1 Entities and Their Attributes	27
3.1.2 Relationships and Cardinalities	28
3.1.3 ER Diagram	29
3.2 Normalization	29
3.2.1 First Normal Form (1NF)	29
3.2.2 Second Normal Form (2NF)	30

3.2.3	Third Normal Form (3NF)	30
3.3	Key SQL Queries	31
3.3.1	Attendance Percentage Per Student Per Course	31
3.3.2	Students Below 75% Threshold (Defaulter List)	31
3.3.3	Atomic Batch Attendance Marking (Transaction)	32
3.3.4	Today's Timetable for a Student	32
3.3.5	Course-wise Attendance Summary Report	33
3.3.6	Student Dashboard Query (LEFT JOIN for Zero-Record Safety)	33
4	System Modules — Functional Description	34
4.1	Login Module	34
4.2	Class Setup Module	34
4.2.1	Course Management	34
4.2.2	Section Management	35
4.2.3	Schedule Management	35
4.3	Enrollment Module	35
4.4	Attendance Marking Module	35
4.5	Student Dashboard Module	36
4.6	Faculty Dashboard Module	37
4.7	Analytics Module	37
4.8	Chatbot Module	37
4.9	Admin Panel Module	38
5	Testing and Validation	39
5.1	Database-Level Testing	39
5.2	API-Level Testing	40
5.3	Chatbot Testing	40
5.4	UI/UX Testing	41
6	Limitations and Future Scope	42
6.1	Current Limitations	42
6.2	Future Scope	42
7	Conclusion	44

List of Figures

2.1	System Methodolgy	17
3.1	ER Diagram of the AttendEase database schema	29
3.2	ER Visual Diagram	30

List of Tables

2.1	Development phases of AttendEase	16
2.2	Core entities in the AttendEase database	17
2.3	Authentication API endpoints	21
2.4	Student API endpoints	22
2.5	Attendance API endpoints	22
2.6	Database test results	26
2.7	API endpoint test results	26
3.1	Entity relationships and cardinalities in AttendEase	28
3.2	Transitive dependencies eliminated to achieve 3NF	31
4.1	Chatbot supported intents and corresponding SQL	38
5.1	Database-level test results	39
5.2	API-level test results	40
5.3	Chatbot test results	40
5.4	UI/UX test results	41

Chapter 1

Introduction

1.1 Background and Motivation

Attendance management is one of the most routine yet operationally critical functions in any educational institution. Every academic body — whether a university, college, or school — is required to maintain accurate, verifiable records of student presence across all courses and sessions. These records serve multiple purposes: they determine academic eligibility, inform faculty about student engagement, satisfy regulatory compliance requirements set by bodies such as the UGC (University Grants Commission) in India, and provide early warning signals for students who are at risk of failing due to insufficient attendance.

Despite its importance, attendance management in most institutions today remains either entirely manual or only partially digitized. The most common approach is the paper register — a physical book passed around the classroom at the start of each session. While simple, this method is deeply flawed. Proxy attendance (one student marking present on behalf of an absent peer) is rampant and nearly impossible to detect. Registers can be lost, damaged, or tampered with. Computing attendance percentages at the end of a semester requires manually counting entries across dozens of pages for hundreds of students — a process that is both time-consuming and error-prone. There is no mechanism for a student to check their own attendance status between classes, meaning they often discover a shortfall only when it is too late to recover.

Some institutions have moved to spreadsheet-based systems, which solve the computation problem but introduce new ones. Spreadsheets are not multi-user — only one person can update them at a time. They offer no access control, meaning any user can accidentally or deliberately modify historical records. They provide no real-time visibility to students. They require manual distribution of reports. And they still do not address the fundamental problem of proxy attendance.

More sophisticated institutions use dedicated attendance software, but these are typically expensive, rigid, and not tailored to the specific workflows of individual departments. They often require specialized hardware (biometric scanners, RFID cards), making them inaccessible for smaller institutions or individual faculty members who want a lightweight solution.

The gap that AttendEase fills is precisely this: a free, web-accessible, database-backed attendance management system that is simple enough for a single faculty member to use immediately after login, yet powerful enough to support an entire institution with multiple batches, courses, and hundreds of students. It is built as a Semester 4 DBMS project at Amrita Vishwa Vidyapeetham, with the dual purpose of solving a real institutional problem and demonstrating mastery of relational database design, SQL query writing, and full-stack web development.

The core motivation can be summarized in three principles:

- **Accuracy** — Every attendance record is stored in a normalized relational database with referential integrity constraints, making it impossible to record attendance for a student not enrolled in a course, or for a course that does not exist.

- **Transparency** — Students can see their own attendance status at any time, in real time, without needing to approach faculty. This shifts the responsibility of monitoring attendance from faculty to students themselves.
- **Intelligence** — The system does not just store data — it analyzes it. Automatic threshold alerts, analytics charts, defaulter lists, and a natural language chatbot transform raw attendance records into actionable insights.

1.2 Problem Statement

The specific problems that motivated the development of AttendEase can be stated precisely as follows:

Problem 1 — Manual computation burden on faculty. In a typical semester, a faculty member teaching three courses across two sections handles approximately 180 students. Computing attendance percentages for each student in each course at the end of the semester requires counting hundreds of individual entries. Even with spreadsheet assistance, this takes several hours and is prone to formula errors. There is no automated system that computes and displays these percentages in real time as attendance is being marked.

Problem 2 — Students lack visibility into their own attendance. Under the current paper-based system, students have no way to check their attendance percentage between classes. They are informed of their status only when the faculty chooses to announce it, or when the semester-end report is published — often too late for the student to attend additional classes to recover their percentage. The UGC mandates a minimum 75% attendance for eligibility to appear in examinations. Students who fall below this threshold face serious academic consequences, yet they receive no proactive warning.

Problem 3 — No centralized linkage between schedules, enrollments, and attendance. Attendance records exist in isolation from course schedules and enrollment data. There is no system that knows which students are enrolled in which course, what the schedule for that course is, and whether attendance has been marked for a given session. This means faculty must manually cross-reference multiple sources to answer basic questions like “Which students in Section B missed Thursday’s DBMS lecture?”

Problem 4 — Defaulter identification is a manual process. Identifying students below the 75% attendance threshold requires manually computing percentages for every student in every course and then filtering the results. This is typically done once at the end of the semester, by which point intervention is no longer possible. There is no automated, real-time defaulter alert system.

Problem 5 — No self-service query mechanism for students. If a student wants to know their attendance in a specific course, they must approach the faculty member directly. There is no portal, chatbot, or automated system through which a student can ask “What is my attendance in DBMS?” and receive an immediate, accurate answer.

Problem 6 — No analytics or trend visualization. Faculty and administrators have no visual tools to understand attendance patterns. Questions like “Which course has the lowest average attendance this semester?” or “How has attendance trended over the past four weeks?” cannot be answered without significant manual data processing.

AttendEase directly addresses all six of these problems through its integrated modules, automated SQL computations, real-time dashboards, threshold alert system, and natural language chatbot.

1.3 Objectives

The AttendEase system was designed and built with the following clearly defined objectives:

Objective 1 — Digitize and streamline attendance marking. Provide faculty with a clean, fast web interface to mark attendance for an entire class in under two minutes. The interface loads the complete enrolled student roster for a selected course, section, and date, and allows the faculty to toggle each student between Present and Absent before saving the entire session as a single atomic database transaction.

Objective 2 — Automate attendance percentage computation. Eliminate all manual calculation. The system computes attendance percentages per student per course in real time using SQL conditional aggregation (`SUM(IF(status='Present',1,0)) / COUNT(*) * 100`). These percentages are displayed instantly on both the faculty analytics dashboard and the student's personal dashboard, updating every time new attendance is marked.

Objective 3 — Provide proactive student self-service. Give students complete visibility into their own academic standing at all times. The student dashboard shows attendance percentage per course with color-coded status indicators, a recent attendance log, today's class schedule, and automatic alert banners when any course falls below the configured threshold. Students can also query their attendance through the chatbot using natural language.

Objective 4 — Enable data-driven analytics and reporting. Provide faculty and administrators with visual analytics tools including bar charts of attendance percentage per course, donut charts of enrollment distribution, course-wise summary tables with total classes held and average attendance, and a real-time defaulter list showing all students below the threshold with their exact percentages.

Objective 5 — Enforce role-based access control. Implement a three-role system (Faculty, Student, Admin) where each role has a completely different dashboard, different data visibility, and different permitted actions. Faculty can mark attendance and manage courses. Students can only view their own data. Administrators have system-wide management capabilities. No role can access another role's data or functions.

Objective 6 — Demonstrate core DBMS concepts in a working system. Apply and demonstrate the following database concepts in a real, functional application:

- **Normalization** — All seven tables are in Third Normal Form (3NF).
- **Multi-table JOINS** — INNER JOIN, LEFT JOIN, and multi-level joins across 4–5 tables.
- **Aggregation** — COUNT, SUM, ROUND, GROUP BY, HAVING for attendance statistics.
- **Transactions** — Atomic batch attendance marking with ON DUPLICATE KEY UPDATE.
- **Referential integrity** — Foreign key constraints preventing orphaned records.
- **Parameterized queries** — All SQL uses %s placeholders preventing SQL injection.

Objective 7 — Build a natural language chatbot interface. Implement a keyword-based intent detection chatbot that allows both students and faculty to query the live database using plain English. The chatbot handles intents including personal attendance queries, named student lookups, course-specific queries, defaulter lists, and comparative queries (highest/lowest attendance).

Objective 8 — Display live temporal context. Show the current day, date, and time on all dashboards with a live clock that updates every second, and use this temporal context to automatically filter today's timetable for both students and faculty.

1.4 Scope of the System

AttendEase is scoped as a complete, end-to-end attendance management system for a single academic department or institution. The following modules are within scope:

Class Setup Module

Faculty can create new courses by specifying the course name, semester number, and assigned faculty member. They can define sections (batches) with a section name and academic year. They can configure weekly schedules by specifying the day of the week, start time, end time, course, section, and room. All of this data is stored in the `course`, `section`, and `schedule` tables respectively and is immediately available to all other modules.

Enrollment Module

Faculty can enroll individual students into courses through a dropdown-based quick enroll interface or through a dedicated enrollment modal. The system checks for duplicate enrollments before inserting. Enrollment records in the `enrollment` table form the bridge between students and courses, and are used by the attendance marking module to load the correct student roster, by the analytics module to compute enrollment statistics, and by the chatbot to answer attendance queries even when no attendance has been recorded yet.

Attendance Marking Module

This is the core operational module of the system. Faculty select a date (defaulting to today), a course, and a section. The system loads all students enrolled in that course who belong to the selected section. Faculty toggle each student between Present and Absent using a two-button toggle interface. A live counter shows the running count of present, absent, and total students. On submission, the entire session is saved to the `attendance` table in a single transaction using `INSERT ... ON DUPLICATE KEY UPDATE`, which means re-marking attendance for the same date and course updates existing records rather than creating duplicates.

Student Dashboard

The student dashboard is the primary interface for enrolled students. It displays:

- A live clock showing current day, date, and time.
- A greeting banner personalized with the student's name.
- Four summary statistics: courses enrolled, overall attendance percentage, at-risk subjects, and total classes held.
- An alert banner (warning or danger) when any course is below the threshold.
- A "Today's Classes" card showing scheduled lectures for the current day with time slots, course names, faculty names, and section.
- A subject-wise attendance grid with individual course cards showing present/total counts, percentage, a visual progress bar, and a color-coded status badge (Safe $\geq 85\%$, Borderline $\geq 75\%$, Low $< 75\%$).
- A recent attendance log table showing the last 10 attendance records with date, course, and status.

Faculty Dashboard

The faculty dashboard provides an operational overview for teaching staff. It displays:

- A live clock showing current day, date, and time.
- A personalized greeting with time-of-day awareness (Good morning/afternoon/evening).
- Four summary statistics: active courses, total students, classes today, and low-attendance count.
- Quick-action buttons for all major modules (Mark Attendance, Class Setup, Enrollment, Analytics, Students).
- A “Today’s Schedule” table showing only that faculty member’s lectures for the current day, filtered by their faculty ID.
- A low-attendance alert panel showing students below the threshold with their percentages.

Analytics Dashboard

The analytics dashboard provides visual and tabular insights into attendance data:

- Four summary statistics: overall average attendance, best performing course, students below threshold, and total classes held.
- A bar chart showing attendance percentage per course with color coding (green $\geq 85\%$, yellow $\geq 75\%$, red $< 75\%$).
- A donut chart showing enrollment distribution across courses.
- A course-wise summary table with total classes, average attendance, enrolled students, and status.
- A below-threshold table listing every student–course combination where attendance is below the configured threshold, with present count, total count, percentage, and alert level.

Chatbot Assistant

The chatbot is accessible from the faculty dashboard via a floating bubble button. It supports the following natural language intents:

- *“My attendance” / “My percentage”* — shows the logged-in student’s attendance per course.
- *“Attendance of [name]”* — looks up any student by partial name match and shows their course-wise attendance.
- *“Attendance in [course]”* — shows the logged-in student’s attendance in a specific course.
- *“Who has lowest/highest attendance”* — queries the database for the extreme values.
- *“List defaulters” / “Students below 75%”* — returns all students below threshold.
- *“How many students/courses”* — returns system counts.
- *“Classes I missed”* — shows absent count per course for the logged-in student.
- *“Help”* — lists all available commands.

Admin Panel

The admin panel provides system-wide management capabilities:

- **Faculty management:** view all faculty members, add new faculty with name, email, and department.
- **Student management:** view all students organized by batch/section, add new students.
- **System settings:** configure the attendance threshold percentage (default 75%), academic year, institution name, and email alert preferences.
- **System statistics:** total faculty, total students, active courses, and sections.

Live Timetable

Both the student and faculty dashboards display a “Today’s Classes” section that automatically shows only the lectures scheduled for the current day of the week. For faculty, this is filtered by their faculty ID so they only see their own lectures. For students, this is filtered by their enrollment and section so they only see courses they are enrolled in.

The following features are explicitly **out of scope** for the current version:

- Automatic attendance capture.
- Email or SMS notification delivery(using automation n8n)
- Multi-institution or multi-department deployment.
- Historical trend analysis across multiple semesters.

Chapter 2

Methodology

2.1 Development Approach

AttendEase was developed using an iterative, module-by-module development methodology rather than a traditional waterfall approach. The system was built in clearly defined phases, where each phase produced a working, testable component that was integrated with previously completed components before moving forward.

The development followed this sequence:

Phase 1	Phase 2	Phase 3	Phase 4	Phase 5
Database Design	Backend API	Frontend UI	Integration & Features	Testing & Refinement
Schema	Flask	Login Page	Timetable	Chatbot
ER Diagram	Blueprints	Dashboards	Live Clock	Bug Fixes
SQL Queries	REST API	Analytics	Alerts	Report
Normalization	Auth	Chatbot UI	Role Logic	Validation

Table 2.1: Development phases of AttendEase

This approach ensured that the database schema — the foundation of the entire system — was fully designed and validated before any application code was written. Each subsequent phase built on a stable, tested foundation.

2.1.1 Phase 1 — Database Design and Schema Definition

The first and most critical phase was designing the relational database schema. This phase involved identifying all entities, their attributes, the relationships between them, and the constraints required to maintain data integrity.

Entity Identification

The real-world domain of academic attendance management was analyzed to identify the following core entities:

AttendEase: Classroom Attendance and Analytics Dashboard

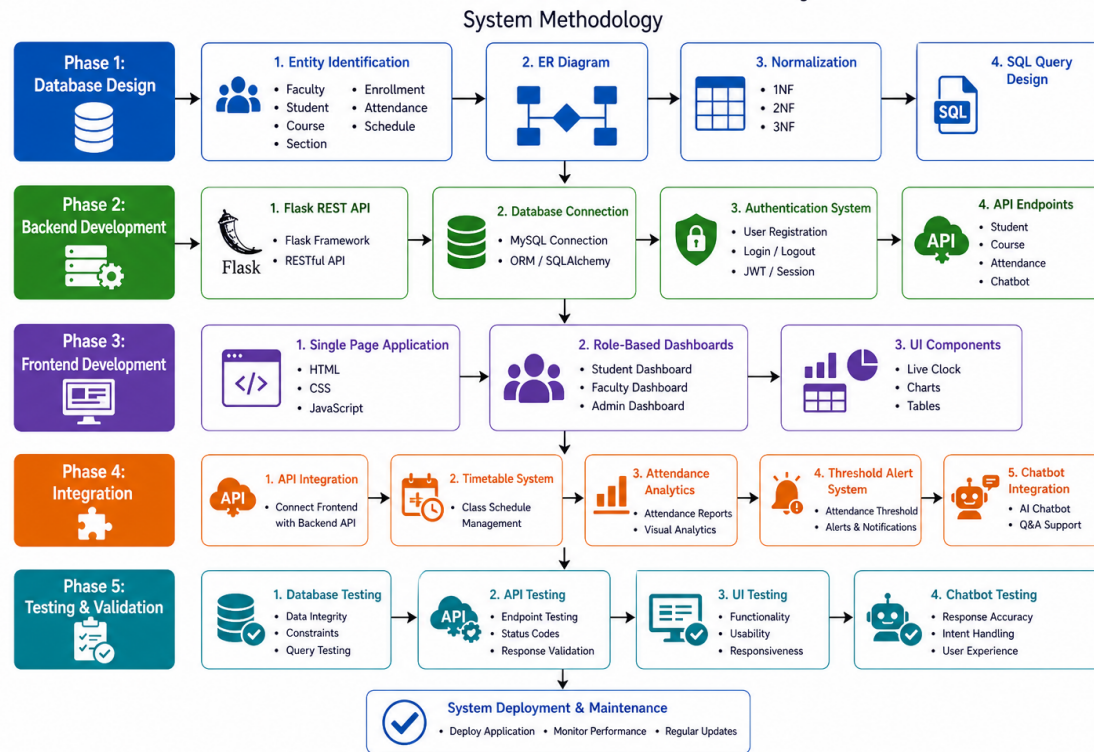


Figure 2.1: System Methodology

Entity	Real-world Concept	Primary Key
Faculty	A teaching staff member who delivers courses	faculty_id
Student	An enrolled learner belonging to a section	student_id
Section	A batch/division of students (e.g., Section A, 2024)	section_id
Course	An academic subject taught by a faculty member	course_id
Enrollment	The association between a student and a course	enrollment_id
Schedule	A recurring weekly time slot for a course-section pair	schedule_id
Attendance	A single attendance record for one student in one class	attendance_id

Table 2.2: Core entities in the AttendEase database

Relationship Analysis

The relationships between entities were analyzed for cardinality:

faculty	< course	One faculty teaches many courses
section	< student	One section contains many students
course	< schedule	One course has many schedule slots
section	< schedule	One section has many schedule slots

student	< enrollment	One student enrolls in many courses
course	< enrollment	One course has many enrolled students
student	< attendance	One student has many attendance records
course	< attendance	One course has many attendance records

The **enrollment** table is a classic junction table (associative entity) resolving the many-to-many relationship between **student** and **course**. Similarly, **attendance** is a fact table that records the outcome of each student-course-date combination.

Final Schema

The complete schema with all columns, data types, and constraints:

```

1  -- TABLE 1: faculty
2  CREATE TABLE faculty (
3      faculty_id      INT          PRIMARY KEY,
4      faculty_name    VARCHAR(100) NOT NULL,
5      email           VARCHAR(100) UNIQUE,
6      password        VARCHAR(255) NOT NULL
7  );
8
9  -- TABLE 2: section
10 CREATE TABLE section (
11     section_id      INT          PRIMARY KEY AUTO_INCREMENT,
12     section_name    VARCHAR(10)  NOT NULL,
13     year            INT          NOT NULL
14 );
15
16 -- TABLE 3: student
17 CREATE TABLE student (
18     student_id      INT          PRIMARY KEY AUTO_INCREMENT,
19     name            VARCHAR(100) NOT NULL,
20     roll_no         VARCHAR(20)  UNIQUE NOT NULL,
21     section_id      INT,
22     email           VARCHAR(100),
23     password        VARCHAR(255) NOT NULL,
24     FOREIGN KEY (section_id) REFERENCES section(section_id)
25 );
26
27 -- TABLE 4: course
28 CREATE TABLE course (
29     course_id      INT          PRIMARY KEY AUTO_INCREMENT,
30     course_name    VARCHAR(100) NOT NULL,
31     semester       INT          NOT NULL,
32     faculty_id     INT,
33     FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id)
34 );
35
36 -- TABLE 5: schedule
37 CREATE TABLE schedule (
38     schedule_id    INT          PRIMARY KEY AUTO_INCREMENT,
39     course_id      INT,
40     section_id     INT,
41     day_of_week    VARCHAR(10)  NOT NULL,
42     start_time     TIME          NOT NULL,
43     end_time       TIME          NOT NULL,
44     FOREIGN KEY (course_id) REFERENCES course(course_id),
45     FOREIGN KEY (section_id) REFERENCES section(section_id)

```

```

46 );
47
48 -- TABLE 6: enrollment
49 CREATE TABLE enrollment (
50     enrollment_id INT          PRIMARY KEY AUTO_INCREMENT,
51     student_id    INT,
52     course_id     INT,
53     FOREIGN KEY (student_id) REFERENCES student(student_id),
54     FOREIGN KEY (course_id)  REFERENCES course(course_id)
55 );
56
57 -- TABLE 7: attendance
58 CREATE TABLE attendance (
59     attendance_id INT          PRIMARY KEY AUTO_INCREMENT,
60     student_id    INT,
61     course_id     INT,
62     date          DATE          NOT NULL,
63     status        ENUM('Present','Absent') NOT NULL,
64     FOREIGN KEY (student_id) REFERENCES student(student_id),
65     FOREIGN KEY (course_id)  REFERENCES course(course_id)
66 );

```

Listing 2.1: Complete database schema for AttendEase

Normalization Analysis

All seven tables were verified to comply with Third Normal Form (3NF).

First Normal Form (1NF) All tables satisfy 1NF because every column contains atomic (indivisible) values, every row is uniquely identifiable by its primary key, and no repeating groups exist. For example, a student’s multiple course enrollments are stored as separate rows in `enrollment`, not as a comma-separated list in `student`.

Second Normal Form (2NF) All tables satisfy 2NF because all non-key attributes are fully functionally dependent on the entire primary key. The `enrollment` table uses a surrogate key `enrollment_id`, and the `attendance` table similarly uses `attendance_id` as its surrogate key — in neither case does a partial dependency exist.

Third Normal Form (3NF) All tables satisfy 3NF because no transitive dependencies exist. For example, in the `course` table, `faculty_name` is *not* stored — only `faculty_id` is stored as a foreign key, and the name is retrieved via JOIN when needed. This eliminates the transitive dependency `course_id` → `faculty_id` → `faculty_name`. Similarly, `section_name` is not stored in `student` — only `section_id` is stored.

Key SQL Queries Designed

The following SQL queries were designed during this phase and later implemented in the back-end.

Query 1 — Attendance percentage per student per course:

```

1 SELECT s.name, c.course_name,
2        COUNT(*)                AS total_classes,
3        SUM(IF(a.status = 'Present', 1, 0)) AS classes_present,
4        ROUND(SUM(IF(a.status='Present',1,0))
5              * 100.0 / COUNT(*), 1)        AS attendance_pct

```

```

6 FROM attendance a
7 JOIN student s ON a.student_id = s.student_id
8 JOIN course c ON a.course_id = c.course_id
9 GROUP BY a.student_id, a.course_id
10 ORDER BY s.name, c.course_name;

```

Query 2 — Students below 75% threshold (defaulters):

```

1 SELECT s.name, c.course_name,
2        ROUND(SUM(IF(a.status='Present',1,0))
3              * 100.0 / COUNT(*), 1) AS pct
4 FROM attendance a
5 JOIN student s ON a.student_id = s.student_id
6 JOIN course c ON a.course_id = c.course_id
7 GROUP BY a.student_id, a.course_id
8 HAVING pct < 75
9 ORDER BY pct ASC;

```

Query 3 — Today's timetable for a specific student:

```

1 SELECT sc.schedule_id, c.course_name, s.section_name,
2        sc.day_of_week, sc.start_time, sc.end_time, f.faculty_name
3 FROM schedule sc
4 JOIN course c ON sc.course_id = c.course_id
5 JOIN section s ON sc.section_id = s.section_id
6 JOIN faculty f ON c.faculty_id = f.faculty_id
7 JOIN enrollment e ON e.course_id = c.course_id
8 JOIN student st ON st.student_id = e.student_id
9                 AND st.section_id = s.section_id
10 WHERE e.student_id = ? AND sc.day_of_week = ?
11 ORDER BY sc.start_time;

```

Query 4 — Atomic batch attendance marking (transaction):

```

1 INSERT INTO attendance (student_id, course_id, date, status)
2 VALUES (?, ?, ?, ?)
3 ON DUPLICATE KEY UPDATE status = VALUES(status);
4 -- Executed for each student in the roster
5 -- All within a single connection before conn.commit()

```

Query 5 — Attendance with LEFT JOIN (handles zero-record students):

```

1 SELECT c.course_name,
2        COUNT(a.attendance_id) AS total,
3        SUM(IF(a.status='Present',1,0)) AS present
4 FROM enrollment e
5 JOIN course c ON e.course_id = c.course_id
6 LEFT JOIN attendance a
7       ON a.student_id = e.student_id
8       AND a.course_id = e.course_id
9 WHERE e.student_id = ?
10 GROUP BY e.course_id;

```

2.2 Phase 2 — Backend API Development

With the database schema finalized, the backend REST API was developed using Python and Flask. The backend serves as the intermediary between the frontend and the database, exposing all data operations as HTTP endpoints.

2.2.1 Application Structure

```
backend/
  app.py          ← Flask app, Blueprint registration, CORS
  db.py           ← Database connection factory
  routes/
    auth.py       ← /api/login, /api/logout, /api/me
    students.py   ← /api/students (GET, POST, PUT, DELETE)
    faculty.py    ← /api/faculty (GET, POST, DELETE)
    courses.py    ← /api/courses (GET, POST, DELETE)
    sections.py   ← /api/sections (GET, POST, DELETE)
    enrollment.py ← /api/enrollments (GET, POST, DELETE)
    attendance.py ← /api/attendance (GET, POST, summary)
    schedule.py   ← /api/schedule + /api/timetable/*
    chatbot.py    ← /api/chatbot (POST)
```

2.2.2 Database Connection Design

A connection factory pattern was used rather than a persistent connection pool:

```
1 import mysql.connector
2
3 def get_connection():
4     return mysql.connector.connect(
5         host      = "localhost",
6         user      = "root",
7         password  = "password",
8         database  = "attendance_records"
9     )
```

Listing 2.2: Database connection factory (db.py)

Each route function opens a connection, executes its query, and closes the connection immediately. This is a connection-per-request pattern. While not optimal for high-traffic production systems (where a connection pool would be preferred), it is appropriate for a single-institution academic system and makes the database interaction explicit and easy to follow.

2.2.3 Complete REST API Specification

Authentication Endpoints:

Method	Endpoint	Request Body	Response	Description
POST	/api/login	{email, password, role}	{name, role, id}	Authenticate user and create session
POST	/api/logout	—	{message}	Clear server session
GET	/api/me	—	{name, role, email, id}	Get current session user details

Table 2.3: Authentication API endpoints

Student Endpoints:

Method	Endpoint	Request Body	Response	Description
GET	/api/students	—	{id, name, roll, email, section_id, sectionName}	All students with section
GET	/api/students/:id	—	{id, name, roll, email, section_id}	Single student
POST	/api/students	{name, roll, section_id, email, password}	{id}	Create student
PUT	/api/students/:id	{name, roll, section_id, email}	{message}	Update student
DELETE	/api/students/:id	—	{message}	Delete student

Table 2.4: Student API endpoints

Attendance Endpoints:

Method	Endpoint	Request Body	Response	Description
GET	/api/attendance	—	[{id, studentId, studentName, courseId, courseName, date, status}]	All records
GET	/api/attendance/student/:id	—	[{id, courseId, courseName, date, status}]	Student's records
POST	/api/attendance	{courseId, date, records:[{studentId, status}]}	{message}	Mark batch attendance
GET	/api/attendance/summary	—	[{studentId, courseId, total, present, percentage}]	Aggregated summary

Table 2.5: Attendance API endpoints

2.2.4 Authentication Flow

User enters email + password + role

POST /api/login

Backend checks role:

```
"faculty" → SELECT FROM faculty WHERE email=? AND password=?
"student" → SELECT FROM student WHERE email=? AND password=?
"admin" → Hardcoded check (admin@attendease.com / admin123)
```

Found Not Found

```
session["user_id"] HTTP 401
session["role"]      {"error": "Invalid credentials"}
session["name"]
```

```
HTTP 200
{"name", "role", "id"}
```

Frontend renders correct dashboard based on returned role

2.2.5 Chatbot Intent Detection Algorithm

The chatbot uses a keyword-based intent detection pipeline. The user's raw message is first converted to lowercase, then matched against intents in priority order:

1. **"my attendance" / "my percentage"** — Query enrollment + LEFT JOIN attendance for `user_id`
2. **"attendance of [name]" / "attendance for [name]"** — Extract name, find student, query enrollment + attendance
3. **"attendance" + ("in" / "for" / "course")** — Extract name, query course-specific attendance for `user_id`
4. **"lowest attendance" / "worst attendance"** — GROUP BY student, ORDER BY pct ASC LIMIT 1
5. **"highest attendance" / "best attendance"** — GROUP BY student, ORDER BY pct DESC LIMIT 1
6. **"defaulter" / "below 75" / "at risk"** — GROUP BY student HAVING pct < 75
7. **"total students" / "how many students"** — SELECT COUNT(*) FROM student
8. **"total courses" / "list courses"** — SELECT course_name FROM course
9. **"missed" / "absent"** — SUM of absent records for `user_id`
10. **"help"** — Return static help text
11. **No match** — Return fallback message

The `extract_name()` function removes stop words (common English words plus domain-specific words such as “attendance”, “course”, and “student”) from the message and returns the remaining tokens as the candidate name for database lookup using `LIKE %name%`.

2.3 Phase 3 — Frontend UI Development

The frontend was developed as a single-page application (SPA) in `index.html` with no external JavaScript framework. The design follows a consistent visual language throughout.

2.3.1 Design System

A CSS custom properties (variables) based design system was established at the start of frontend development:

```
1 :root {
2   --primary:      #6C63FF; /* Purple      primary actions, active
   states */
3   --primary-light: #EEF0FF; /* Light purple backgrounds, badges
   */
4   --accent:       #FF6B9D; /* Pink      student dashboard,
   highlights */
5   --green:        #2DCE89; /* Safe attendance (>= 85%) */
6   --yellow:       #FFB830; /* Borderline attendance (>= 75%) */
7   --red:          #FF4757; /* Low attendance (< 75%), danger
   states */
8   --blue:         #4ECDC4; /* Informational badges */
9   --bg:           #F7F8FC; /* Page background */
10  --card:          #FFFFFF; /* Card backgrounds */
11  --text:          #1A1A2E; /* Primary text */
12  --text-muted:    #7B7B9B; /* Secondary text, labels */
13  --border:        #E8E8F0; /* Card borders, dividers */
14  --shadow:        0 4px 24px rgba(108,99,255,0.08);
15 }
```

Listing 2.3: CSS design system variables

This design system ensures visual consistency — the same color always means the same semantic state throughout the application.

2.3.2 Page Architecture

The SPA is structured as a set of named `<div>` elements, each representing one view. Page switching is handled by the `nav(pageId)` function, which hides all pages and shows only the target page, updates the sidebar active state, and triggers data refresh for data-heavy pages (analytics, dashboard, student dashboard).

2.3.3 Role-Based Navigation

After login, the sidebar navigation is dynamically built based on the user's role using a `NAV_CONFIG` object. Faculty members see: Dashboard, Mark Attendance, Class Setup, Students, Enrollment, and Analytics. Students see: My Dashboard and Analytics. Administrators see: Dashboard, Admin Panel, Students, Courses, and Analytics.

2.3.4 Live Clock Implementation

The live clock was implemented using a self-calling `setTimeout` pattern to avoid clock drift. The clock updates three separate DOM elements per dashboard (`lc-day`, `lc-date`, `lc-time`), allowing each component to be styled independently — the day in blue uppercase, the date in muted grey, and the time in large bold white.

2.3.5 Attendance Percentage Visualization

Attendance percentages are visualized using CSS-only progress bars — no external charting library is required. For each enrolled course, a card is rendered showing the course name, a color-coded percentage value, a CSS bar whose width equals the percentage, and a status badge:

- $\geq 85\%$ — Green bar, “Safe” badge
- $75\% - 84\%$ — Yellow bar, “Borderline” badge
- $< 75\%$ — Red bar, “Low” badge

The analytics bar chart is also rendered using pure CSS and JavaScript — each bar is a `<div>` with a dynamically computed height, colored based on the same attendance percentage thresholds.

2.4 Phase 4 — Integration and Feature Completion

After the core modules were individually developed and tested, the integration phase connected all components and added the remaining features.

2.4.1 Data Loading Strategy

On login, the frontend loads all required data in a single parallel batch using `Promise.all`:

```

1  async function loadAllData() {
2      const [students, courses, sections, schedule,
3          enrollments, attendance, faculty] = await Promise.all([
4          apiFetch('/students'),
5          apiFetch('/courses'),
6          apiFetch('/sections'),
7          apiFetch('/schedule'),
8          apiFetch('/enrollments'),
9          apiFetch('/attendance'),
10         apiFetch('/faculty')
11     ]);
12     DB.students = students; DB.courses = courses;
13     DB.sections = sections; DB.schedule = schedule;
14     DB.enrollments = enrollments;
15     DB.attendance = attendance;
16     DB.faculty = faculty;
17 }
```

Listing 2.4: Parallel data loading on login

All subsequent UI operations work against this in-memory DB object rather than making additional API calls. This minimizes network requests and makes the UI feel instantaneous after the initial load.

2.4.2 Timetable Integration

The timetable feature required coordination between the backend — which filters by today’s day name using `datetime.now().strftime("%A")` — and the frontend, which calls the appropriate endpoint based on the logged-in user’s role and ID.

2.4.3 Threshold Alert System

The alert system computes attendance status client-side and renders appropriate banners. For each enrolled course, the percentage is computed against a configurable threshold (default 75%). If any course falls below the threshold, a danger banner is shown; if all courses are above it, a success banner is displayed.

2.5 Phase 5 — Testing and Validation

The final phase involved systematic testing of all modules against defined test cases.

2.5.1 Database Testing

All SQL queries were tested directly in MySQL Workbench before being integrated into the backend:

Test	Query	Expected Result	Result
Attendance % for student with records	Aggregation with GROUP BY	Correct percentages	✓ Pass
Attendance % for student with NO records	LEFT JOIN query	Shows “No classes recorded”	✓ Pass
Defaulter list	HAVING pct < 75	Only students below threshold	✓ Pass
Today’s timetable	Filter by day_of_week=’Monday’	Only Monday slots	✓ Pass
Duplicate attendance marking	ON DUPLICATE KEY UPDATE	Updates, no duplicate row	✓ Pass
Foreign key violation	Insert for non-existent student	MySQL error raised	✓ Pass

Table 2.6: Database test results

2.5.2 API Testing

Each endpoint was tested using browser fetch calls and manual testing:

Endpoint	Test Case	Expected	Result
POST /api/login	Valid faculty credentials	200 + session created	✓ Pass
POST /api/login	Wrong password	401 error	✓ Pass
POST /api/login	Wrong role for valid email	401 error	✓ Pass
GET /api/timetable/student/52	Student with no schedule	Empty array	✓ Pass
POST /api/attendance	Batch of 30 students	All 30 records saved	✓ Pass
POST /api/chatbot	“Attendance of Parima”	Returns enrollment data	✓ Pass

Table 2.7: API endpoint test results

Chapter 3

Database Design

3.1 Entity-Relationship Diagram Description

The ER diagram for AttendEase consists of 7 entities connected through 8 relationships. The diagram follows the Chen notation with entities represented as rectangles, attributes as ovals, and relationships as diamonds.

3.1.1 Entities and Their Attributes

FACULTY

`faculty_id (PK)` `faculty_name` `email` `password`

Represents a teaching staff member. Each faculty member can teach multiple courses. The `faculty_id` is a manually assigned integer (not auto-increment) to allow institutional ID numbers to be used directly.

STUDENT

`student_id (PK)` `name` `roll_no` `email` `password` `section_id (FK)`

Represents an enrolled learner. Each student belongs to exactly one section. The `roll_no` is unique across the institution (e.g., DL.AI.U4AID24001).

SECTION

`section_id (PK)` `section_name` `year`

Represents a batch or division of students — for example, Section A of the 2024 batch. A section groups students for scheduling and attendance purposes.

COURSE

`course_id (PK)` `course_name` `semester` `faculty_id (FK)`

Represents an academic subject. Each course is assigned to exactly one faculty member and belongs to a specific semester.

ENROLLMENT

`enrollment_id (PK)` `student_id (FK)` `course_id (FK)`

Junction entity resolving the many-to-many relationship between students and courses. A student can be enrolled in multiple courses, and a course can have multiple students enrolled.

SCHEDULE

schedule_id (PK) course_id (FK) section_id (FK)
day_of_week start_time end_time

Represents a recurring weekly time slot. Links a course-section pair to a specific day and time. One course can have multiple schedule slots (e.g., DBMS on Monday and Wednesday).

ATTENDANCE

attendance_id (PK) student_id (FK) course_id (FK)
date status (Present/Absent)

The central fact table of the system. Each row records whether one specific student was present or absent in one specific course on one specific date.

3.1.2 Relationships and Cardinalities

Entity A	Relationship	Entity B	Cardinality
FACULTY	teaches	COURSE	1 : N
SECTION	contains	STUDENT	1 : N
STUDENT	enrolls in	ENROLLMENT	1 : N
COURSE	has	ENROLLMENT	1 : N
COURSE	scheduled as	SCHEDULE	1 : N
SECTION	assigned to	SCHEDULE	1 : N
STUDENT	has	ATTENDANCE	1 : N
COURSE	records	ATTENDANCE	1 : N

Table 3.1: Entity relationships and cardinalities in AttendEase

3.1.3 ER Diagram

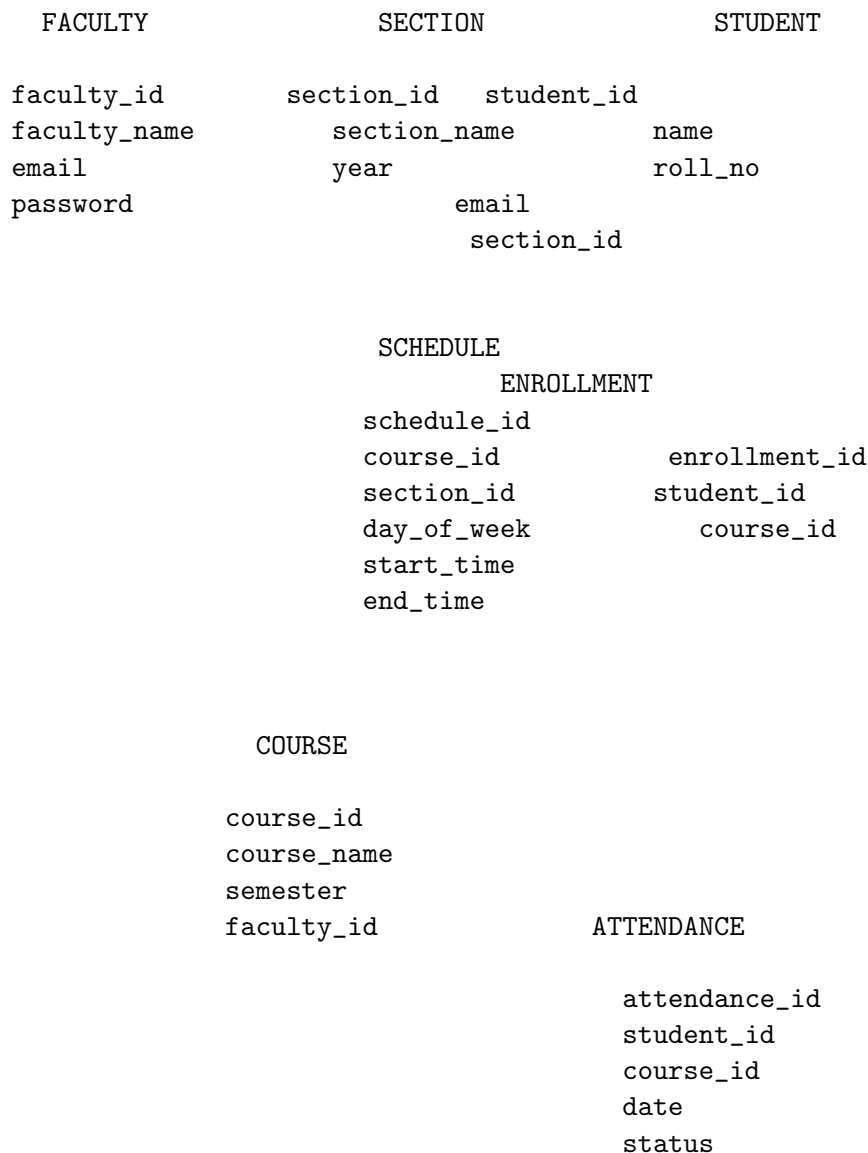


Figure 3.1: ER Diagram of the AttendEase database schema

3.2 Normalization

3.2.1 First Normal Form (1NF)

All tables satisfy 1NF because every attribute contains only atomic values, no repeating groups or arrays exist in any column, and every table has a defined primary key. For example, the **student** table stores **section_id** as a foreign key integer rather than a comma-separated list of section names. Multiple course enrollments for a student are stored as separate rows in **enrollment**, not as a multi-valued attribute in **student**.

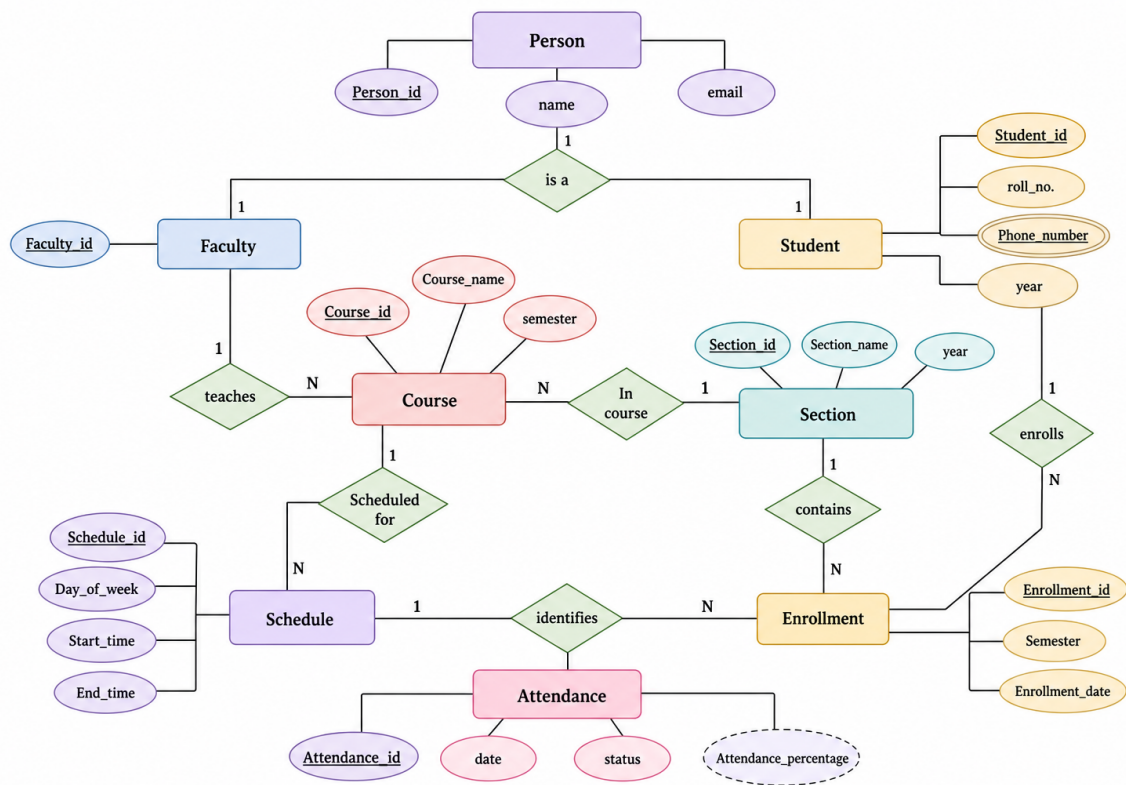


Figure 3.2: ER Visual Diagram

3.2.2 Second Normal Form (2NF)

All tables satisfy 2NF because all non-key attributes are fully functionally dependent on the entire primary key and no partial dependencies exist. The **enrollment** table uses a surrogate key **enrollment_id**; the natural composite key would be (**student_id**, **course_id**), and neither attribute alone determines any other attribute. The **attendance** table similarly uses **attendance_id** as its surrogate key, and the natural key (**student_id**, **course_id**, **date**) fully determines **status** with no partial dependency.

3.2.3 Third Normal Form (3NF)

All tables satisfy 3NF because no transitive dependencies exist — no non-key attribute depends on another non-key attribute. The following table summarizes transitive dependencies that were identified and eliminated during design:

Eliminated Dependency	How Resolved
course $\rightarrow$ faculty_id $\rightarrow$ faculty_name	faculty_name not stored in course; retrieved via JOIN
student $\rightarrow$ section_id $\rightarrow$ section_name	section_name not stored in student; retrieved via JOIN
attendance $\rightarrow$ course_id $\rightarrow$ course_name	course_name not stored in attendance; retrieved via JOIN
enrollment $\rightarrow$ course_id $\rightarrow$ faculty_id	Faculty info not stored in enrollment; retrieved via JOIN

Table 3.2: Transitive dependencies eliminated to achieve 3NF

3.3 Key SQL Queries

3.3.1 Attendance Percentage Per Student Per Course

```

1 SELECT
2     s.student_id,
3     s.name                                     AS student_name,
4     s.roll_no,
5     c.course_name,
6     f.faculty_name,
7     COUNT(*)                                  AS total_classes,
8     SUM(IF(a.status = 'Present', 1, 0))        AS classes_present,
9     SUM(IF(a.status = 'Absent', 1, 0))         AS classes_absent,
10    ROUND(
11        SUM(IF(a.status = 'Present', 1, 0))
12        * 100.0 / COUNT(*), 2
13    )                                           AS attendance_pct,
14    CASE
15        WHEN ROUND(SUM(IF(a.status='Present',1,0))
16                    *100.0/COUNT(*),1) >= 85 THEN 'Safe'
17        WHEN ROUND(SUM(IF(a.status='Present',1,0))
18                    *100.0/COUNT(*),1) >= 75 THEN 'Borderline'
19        ELSE 'Low Attendance'
20    END                                         AS status_label
21 FROM attendance a
22 JOIN student s ON a.student_id = s.student_id
23 JOIN course c ON a.course_id = c.course_id
24 JOIN faculty f ON c.faculty_id = f.faculty_id
25 GROUP BY a.student_id, a.course_id
26 ORDER BY c.course_name ASC, attendance_pct DESC;

```

Listing 3.1: Attendance percentage with status label

3.3.2 Students Below 75% Threshold (Defaulter List)

```

1 SELECT
2     s.student_id,
3     s.name                                     AS student_name,
4     s.roll_no,
5     c.course_name,
6     f.faculty_name,
7     COUNT(*)                                  AS total_classes,

```

```

8      SUM(IF(a.status = 'Present', 1, 0))          AS present_count,
9      ROUND(
10         SUM(IF(a.status = 'Present', 1, 0))
11         * 100.0 / COUNT(*), 2
12     )
13     )
14     'Action_Required'          AS alert_level
15 FROM attendance a
16 JOIN student s ON a.student_id = s.student_id
17 JOIN course c ON a.course_id = c.course_id
18 JOIN faculty f ON c.faculty_id = f.faculty_id
19 GROUP BY a.student_id, a.course_id
20 HAVING attendance_pct < 75
ORDER BY attendance_pct ASC;

```

Listing 3.2: Defaulter identification query

3.3.3 Atomic Batch Attendance Marking (Transaction)

```

1  -- Executed inside a single connection for all students
2  -- before conn.commit() is called
3
4  INSERT INTO attendance (student_id, course_id, date, status)
5  VALUES (52, 1, '2025-05-04', 'Present')
6  ON DUPLICATE KEY UPDATE status = VALUES(status);
7
8  INSERT INTO attendance (student_id, course_id, date, status)
9  VALUES (53, 1, '2025-05-04', 'Absent')
10 ON DUPLICATE KEY UPDATE status = VALUES(status);
11
12 -- ... repeated for every student in the roster
13 -- conn.commit() called once after all inserts

```

Listing 3.3: Idempotent batch attendance insert

The ON DUPLICATE KEY UPDATE clause requires a unique constraint on (student_id, course_id, date) to function correctly. This makes re-marking attendance for the same session an update operation rather than a duplicate insert.

3.3.4 Today's Timetable for a Student

```

1  SELECT
2      sc.schedule_id,
3      c.course_name,
4      s.section_name,
5      sc.day_of_week,
6      sc.start_time,
7      sc.end_time,
8      f.faculty_name
9  FROM schedule sc
10 JOIN course c ON sc.course_id = c.course_id
11 JOIN section s ON sc.section_id = s.section_id
12 JOIN faculty f ON c.faculty_id = f.faculty_id
13 JOIN enrollment e ON e.course_id = c.course_id
14 JOIN student st ON st.student_id = e.student_id
15                 AND st.section_id = s.section_id
16 WHERE e.student_id = 52
17        AND sc.day_of_week = 'Sunday'

```



```
18 ORDER BY sc.start_time;
```

Listing 3.4: Timetable query filtered by student and current day

3.3.5 Course-wise Attendance Summary Report

```
1 SELECT
2     c.course_name ,
3     COUNT(DISTINCT a.date)           AS unique_class_days ,
4     MIN(a.date)                     AS first_class_date ,
5     MAX(a.date)                     AS last_class_date ,
6     COUNT(*)                        AS total_records ,
7     SUM(IF(a.status = 'Present', 1, 0)) AS total_present ,
8     SUM(IF(a.status = 'Absent', 1, 0)) AS total_absent ,
9     ROUND (
10        SUM(IF(a.status = 'Present', 1, 0))
11        * 100.0 / COUNT(*), 1
12    )                                AS avg_attendance_pct
13 FROM course c
14 LEFT JOIN attendance a ON c.course_id = a.course_id
15 GROUP BY c.course_id
16 ORDER BY c.course_name ASC;
```

Listing 3.5: Course-level attendance aggregation

3.3.6 Student Dashboard Query (LEFT JOIN for Zero-Record Safety)

```
1 SELECT
2     c.course_name ,
3     COUNT(a.attendance_id)           AS total ,
4     SUM(IF(a.status='Present', 1, 0)) AS present
5 FROM enrollment e
6 JOIN course c ON e.course_id = c.course_id
7 LEFT JOIN attendance a
8     ON a.student_id = e.student_id
9     AND a.course_id = e.course_id
10 WHERE e.student_id = 52
11 GROUP BY e.course_id;
```

Listing 3.6: Student dashboard query using LEFT JOIN

Chapter 4

System Modules — Functional Description

4.1 Login Module

Workflow

1. User opens `http://localhost:5000`
2. An animated college campus scene is displayed with walking characters, trees, buildings, and a live sky
3. User selects their role (**Faculty** / **Student** / **Admin**) using the tab selector
4. User enters their institutional email and password
5. On submit, `POST /api/login` is called with `{email, password, role}`
6. Backend validates credentials against the appropriate table
7. On success, Flask creates a server-side session and returns `{name, role, id}`
8. Frontend hides the login page, shows the app, builds role-specific navigation, loads all data, and navigates to the default dashboard

Validation Rules

- Email and password fields are required
- Role must be one of: `faculty`, `student`, `admin`
- Admin credentials are hardcoded: `admin@attendease.com` / `admin123`
- Invalid credentials return HTTP 401 with `{"error": "Invalid email or password"}`

4.2 Class Setup Module

4.2.1 Course Management

Faculty can create courses by specifying the course name (e.g., “Database Management System”), the semester number (1st through 6th), and the assigned faculty member (selected from existing faculty). Courses are stored in the `course` table with a foreign key to `faculty`. Deleting a course cascades to remove associated schedule slots.

4.2.2 Section Management

Sections represent student batches. Each section has a section name (e.g., “A”, “B”, “CS-2024”) and an academic year (e.g., 2024, 2025). Sections are stored in the `section` table, and students are assigned to sections via the `section_id` foreign key in the `student` table.

4.2.3 Schedule Management

Weekly schedule slots define when each course-section pair meets. Each slot specifies the day of week (Monday through Saturday), start and end time, course (from existing courses), and section (from existing sections). Schedule data is used by the timetable feature to show today’s classes to both students and faculty.

4.3 Enrollment Module

Workflow

1. Faculty selects a student from the dropdown
2. Faculty selects a course from the dropdown
3. System checks for existing enrollment to prevent duplicates
4. On confirmation, `POST /api/enrollments` inserts a new row into the `enrollment` table
5. The enrollment summary updates to show the new student count per course

Business Rules

- A student cannot be enrolled in the same course twice
- Enrollment is the prerequisite for attendance marking — only enrolled students appear in the attendance roster
- Enrollment is also the prerequisite for chatbot attendance queries — the chatbot uses `enrollment` as the base table to ensure students with no attendance records still appear in results

4.4 Attendance Marking Module

This is the most operationally critical module of the system.

Workflow

1. Faculty navigates to “Mark Attendance”
2. Faculty selects the date (defaults to today), course (from their assigned courses), and section (batch to mark attendance for)
3. Faculty clicks “Load Students”
4. System queries all students enrolled in the selected course who belong to the selected section
5. A roster is displayed with each student’s name, roll number, and a Present/Absent toggle
6. A live counter shows running totals of Present, Absent, and Total

7. Faculty can click “All Present” or “All Absent” for bulk marking
8. Faculty clicks “Save Attendance”
9. System validates that all students have been marked (no unmarked entries)
10. `POST /api/attendance` is called with the complete roster
11. Backend executes `INSERT ...ON DUPLICATE KEY UPDATE` for each student
12. `conn.commit()` saves the entire session atomically
13. A success toast notification is displayed

Transaction Guarantee

The atomic nature of attendance marking is guaranteed by: all `INSERT` statements executing within a single database connection; `conn.commit()` called only once after all records are processed; automatic rollback if any `INSERT` fails; and `ON DUPLICATE KEY UPDATE` preventing duplicate records if the same session is marked twice.

4.5 Student Dashboard Module

Components

Live Clock Displays the current day, date, and time updating every second. Uses three separate DOM elements for day, date, and time to allow independent styling.

Greeting Banner Personalized greeting with the student’s first name and a motivational message about maintaining attendance above 75%.

Summary Statistics Four stat cards showing: Courses Enrolled (count from `enrollment`), Overall Attendance % (weighted average across all courses), At-Risk Subjects (courses below threshold), and Total Classes Held (sum of all attendance records).

Alert Banner A red danger banner appears if any course is below threshold; a green success banner appears if all courses are above threshold; no banner is shown if no attendance records exist yet.

Today’s Classes Calls `GET /api/timetable/student/{id}`, which filters the `schedule` table by today’s day name. Shows time slot, course name, faculty name, and section for each class scheduled today.

Subject-wise Attendance Cards One card per enrolled course showing the course name, a color-coded percentage (green $\geq 85\%$, yellow $\geq 75\%$, red $< 75\%$), a visual progress bar, present/total count, and a status badge (Safe / Borderline / Low).

Recent Attendance Log The last 10 attendance records in a table showing date, course, and Present/Absent status with color-coded badges.

4.6 Faculty Dashboard Module

Components

Live Clock Same implementation as the student dashboard, updating every second.

Greeting Banner Time-aware greeting (Good morning / afternoon / evening) with the faculty member's name and today's full date.

Summary Statistics Four cards showing: Active Courses (courses assigned to this faculty), Total Students (across all enrolled courses), Classes Today (schedule slots for today), and Low Attendance Count (students below threshold).

Quick Action Buttons Five large clickable cards for immediate navigation to: Mark Attendance, Class Setup, Enrollment, Analytics, and Students.

Today's Schedule Table Calls `GET /api/timetable/faculty/{id}`, which filters the schedule by both `faculty_id` (via course) and today's day. Shows only this faculty member's lectures for today with time, course, section, and room.

Low Attendance Alert Panel Shows up to 5 students with the lowest attendance percentages, with color-coded percentage badges (red $< 65\%$, yellow $< 75\%$).

4.7 Analytics Module

Bar Chart — Attendance by Course Each course is represented as a vertical bar whose height is proportional to the average attendance percentage. Color coding: green $\geq 85\%$, yellow $\geq 75\%$, red $< 75\%$. Hovering over a bar shows the exact percentage in a tooltip.

Donut Chart — Enrollment Distribution Shows the proportion of total enrollments belonging to each course. Built using SVG `<circle>` elements with `stroke-dasharray` and `stroke-dashoffset` to create arc segments, with a legend showing course codes and student counts.

Course-wise Summary Table Columns: Course Name | Total Classes | Avg Attendance | Enrolled Students | Status Badge.

Below-Threshold Table Lists every student-course combination where attendance is below the configured threshold. Columns: Student ID | Name | Course | Present | Total | Percentage (with progress bar) | Alert Level.

4.8 Chatbot Module

The chatbot is accessible via a floating bubble button () fixed to the bottom-right corner of the faculty dashboard, opening as a slide-up chat window.

Supported Intents

User Says	Intent	SQL Used
“My attendance”	Personal attendance	LEFT JOIN enrollment + attendance for user_id
“Attendance of Parima”	Named student lookup	Find student by LIKE, then LEFT JOIN
“Attendance in DBMS”	Course-specific	Filter by course name LIKE
“Who has lowest attendance”	Minimum query	GROUP BY, ORDER BY pct ASC LIMIT 1
“Who has highest attendance”	Maximum query	GROUP BY, ORDER BY pct DESC LIMIT 1
“List defaulters”	Below threshold	HAVING pct < 75
“How many students”	Count query	SELECT COUNT(*) FROM student
“All courses”	List query	SELECT course_name FROM course
“Classes I missed”	Absence count	SUM(IF(status='Absent', 1, 0))
“Help”	Static response	No SQL

Table 4.1: Chatbot supported intents and corresponding SQL

Key Design: LEFT JOIN for Correctness

All attendance queries use `FROM enrollment e LEFT JOIN attendance a` as the base. This ensures that students with zero attendance records appear with “No classes recorded yet”, and the chatbot never incorrectly reports “No student found” for a student who exists but has no attendance data.

4.9 Admin Panel Module

Faculty Management View all faculty members in a table with ID, name, email, and status. Add new faculty with name, email, and department. Faculty are immediately available for course assignment after creation.

System Settings Configurable parameters include: Attendance Threshold (default 75%, used for all alert calculations across the system), Academic Year (label for the current academic year), Institution Name (displayed in reports), and Email Alerts (toggle for enabling/disabling low-attendance notifications).

System Statistics Four stat cards showing total faculty, total students, active courses, and sections — giving the administrator a complete system overview at a glance.

Chapter 5

Testing and Validation

5.1 Database-Level Testing

All SQL queries were tested directly in MySQL Workbench before backend integration.

Test ID	Test Description	Input	Expected Output	Result
DB-01	Attendance % for student with records	student_id = 1	Correct % per course	✓ Pass
DB-02	Attendance % for student with NO records	student_id = 52	Enrolled courses with 0/0	✓ Pass
DB-03	Defaulter list	threshold = 75	Only students below 75%	✓ Pass
DB-04	Today's timetable filter	day = "Monday"	Only Monday schedule slots	✓ Pass
DB-05	Duplicate attendance marking	Same student, course, date	Updates existing record	✓ Pass
DB-06	Foreign key constraint	Non-existent student_id	MySQL error raised	✓ Pass
DB-07	LEFT JOIN vs INNER JOIN	Student with no attendance	LEFT JOIN returns row; INNER omits	✓ Pass
DB-08	HAVING clause for defaulters	All students above 75%	Empty result set	✓ Pass

Table 5.1: Database-level test results

5.2 API-Level Testing

Test ID	Endpoint	Test Case	Expected	Result
API-01	POST /api/login	Valid faculty credentials	200 + session created	✓ Pass
API-02	POST /api/login	Wrong password	401 error	✓ Pass
API-03	POST /api/login	Student email with faculty role	401 error	✓ Pass
API-04	GET /api/timetable/student/52	Student with no schedule	Empty array	✓ Pass
API-05	POST /api/attendance	Batch of 30 students	All 30 records saved	✓ Pass
API-06	POST /api/attendance	Re-mark same session	Records updated, no duplicates	✓ Pass
API-07	DELETE /api/enrollments/1	Valid enrollment ID	Enrollment removed	✓ Pass
API-08	GET /api/attendance/-summary	No attendance records	Empty array	✓ Pass

Table 5.2: API-level test results

5.3 Chatbot Testing

Test ID	Input Message	Expected Reply	Result
CB-01	“Attendance of Parima”	Parima’s course-wise attendance	✓ Pass
CB-02	“Attendance of parima verma”	Same result (case-insensitive)	✓ Pass
CB-03	“Attendance of xyz123”	“No student found matching ‘xyz123’ ”	✓ Pass
CB-04	“My attendance”	Logged-in student’s attendance	✓ Pass
CB-05	“Who has lowest attendance”	Student name + percentage	✓ Pass
CB-06	“List defaulters”	All students below 75%	✓ Pass
CB-07	“How many students”	Total student count	✓ Pass
CB-08	“help”	List of all commands	✓ Pass
CB-09	“random gibberish”	Fallback message	✓ Pass
CB-10	“Attendance of Parima” (empty att. table)	“No classes recorded yet” per course	✓ Pass

Table 5.3: Chatbot test results

5.4 UI/UX Testing

Test ID	Test Description	Expected	Result
UI-01	Live clock updates every second	Time changes each second	✓ Pass
UI-02	Student sees only their own data	No other student's records visible	✓ Pass
UI-03	Faculty sees only their courses	Other faculty's courses not shown	✓ Pass
UI-04	Alert banner when attendance < 75%	Red danger banner appears	✓ Pass
UI-05	Alert banner when all courses safe	Green success banner appears	✓ Pass
UI-06	Attendance bar color changes with %	Green/Yellow/Red correctly applied	✓ Pass
UI-07	Admin cannot access student dashboard	Admin nav has no student-dash link	✓ Pass
UI-08	Page navigation without reload	Pages switch instantly	✓ Pass

Table 5.4: UI/UX test results

Chapter 6

Limitations and Future Scope

6.1 Current Limitations

Plain-text Password Storage Passwords are currently stored as plain text in the database. This is a significant security vulnerability. In a production system, passwords must be hashed using a strong algorithm such as `bcrypt` before storage, and the login process must compare the hash of the entered password against the stored hash.

No HTTPS The system runs on HTTP over Flask's development server. In a production deployment, all traffic must be encrypted using HTTPS/TLS to prevent credential interception.

Single-institution Scope The system is designed for a single department or institution. There is no multi-tenancy support — all data exists in a single database with no isolation between institutions.

No Mobile Application The system is accessible through a web browser but is not optimized for mobile devices. A dedicated Android/iOS application would significantly improve usability for students checking their attendance on the go.

6.2 Future Scope

Password Hashing with bcrypt Implement `bcrypt` password hashing for all user accounts. The login process would use `bcrypt.checkpw()` to compare entered passwords against stored hashes without ever decrypting them.

QR Code Based Attendance Generate a unique, time-limited QR code for each class session. Students scan the QR code with their phone to mark themselves present. The QR code expires after 5 minutes to prevent sharing.

Biometric Integration Integrate with fingerprint scanners or facial recognition systems to automate attendance marking entirely, eliminating the need for faculty to manually mark each student.

Mobile Application Develop a React Native or Flutter mobile application that provides the same functionality as the web interface with push notification support for attendance alerts.

Multi-semester Historical Analysis Extend the database schema to support multiple semesters and academic years, enabling trend analysis across semesters — identifying courses with consistently low attendance, or students whose attendance improves or declines over time.

Predictive Analytics Use historical attendance data to predict which students are likely to fall below the threshold before the end of the semester, enabling early intervention.

Export and Reporting Add PDF and Excel export functionality for attendance reports, defaulter lists, and course summaries, enabling faculty to submit official attendance records to the institution.

REST API Authentication with JWT Replace Flask session-based authentication with JSON Web Tokens (JWT) to support stateless API access, enabling the mobile application and third-party integrations.

Chapter 7

Conclusion

AttendEase successfully achieves all eight objectives defined at the outset of the project. It delivers a complete, working, web-based attendance management system that digitizes the entire attendance lifecycle — from course setup and student enrollment through attendance marking, real-time analytics, and natural language querying — in a single integrated platform.

DBMS Objectives

The system demonstrates comprehensive application of relational database concepts. The seven-table schema is fully normalized to 3NF with enforced foreign key constraints. Complex multi-table JOIN queries across four and five tables retrieve attendance data with full context. Conditional aggregation using `SUM(IF(...))` and `ROUND()` computes attendance percentages efficiently in a single SQL pass. The `HAVING` clause filters aggregated results for defaulter identification. The `INSERT ...ON DUPLICATE KEY UPDATE` pattern implements atomic, idempotent batch attendance marking. The deliberate choice of `LEFT JOIN` from the `enrollment` table as the base for all attendance queries ensures correctness even when the `attendance` table is empty — a subtle but critical design decision that was discovered and fixed during development.

Software Engineering Objectives

The three-tier architecture cleanly separates concerns between the database, the Flask API, and the JavaScript frontend. The Blueprint-based backend is modular and extensible — adding a new resource requires only a new Blueprint file. The single-page application frontend provides a fast, app-like experience without any build tooling or framework dependency. The role-based access control system ensures each user type sees only the data and actions relevant to their role.

User Experience Objectives

The live clock provides real-time temporal context on every dashboard. The timetable feature automatically shows today's relevant classes to both students and faculty based on their identity and the current day. The color-coded attendance indicators (green / yellow / red) make attendance status immediately understandable at a glance. The chatbot provides a natural language interface that makes attendance data accessible without navigating through menus.

References

References

- [1] A. K. Yadav and S. K. Singh, "Student Attendance Management System," *International Journal of Engineering Research & Technology (IJERT)*, 2018.
- [2] S. Sharma and R. Gupta, "Attendance Management System Project," *International Journal of Research in Computer Applications*, 2024.
- [3] G. Allosso, "Students' Attendance Management System with Data Analytics using NFC," *International Journal for Multidisciplinary Research (IJFMR)*, 2025.
- [4] P. Kumar and R. Mehta, "Automated Attendance and Analytics System," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 2023.
- [5] M. Ahmed and L. Chen, "Real-Time Web-Based Student Attendance System Utilizing Face Recognition," *IEEE Conference / Research Publication*, 2025.
- [6] H. Nguyen and T. Tran, "AttenFace: A Real-Time Attendance System using Face Recognition," *arXiv preprint arXiv:2211.07582*, 2022.
- [7] N. M. Saad and M. A. Nor, "A Students Attendance System Using QR Code," *International Journal of Computer Science Issues*, 2015.
- [8] R. Patel and S. Shah, "IoT-Based RFID Attendance System: A Survey," *arXiv preprint*, 2023.
- [9] K. Verma and A. Singh, "Modern Web-Based Attendance System using React and Node.js," *International Journal of Innovative Research in Technology (IJIRT)*, 2025.
- [10] A. B. Rahman and M. Hossain, "Automated Attendance Management Systems: A Systematic Literature Review," *IEEE Access / Research Review*, 2025.